TAN TAO UNIVERSITY

SCHOOL OF INFORMATION TECHNOLOGY

TAN TAO UNIVERSITY

FROM KNOWLEDGE TO THE STARS

Report of Final Project

BEER DISTRIBUTION SYSTEM

Course: Introduction to Database System (CS311V)

Fall 2025

GROUP 2

Member 1: Le Chi Tam	**Student code: 2302220**
Member 2: Duong The Hien	**Student code: 2302251**
Member 3: Le Nguyen Quoc Anh	**Student code: 2302283**

Supervisor: Cao Tien Dung, PhD

Github: ***https://github.com/quynhanh6125/CS3011V***

Abstract

This technical report presents the design and implementation of a **Beer Distribution Management System**, a comprehensive database-driven application built to modernize beverage distribution operations. The system integrates MySQL as the database foundation with a FastAPI backend to provide efficient customer management, order processing, inventory tracking, and supplier coordination.

The architecture follows a three-layer design pattern: API Layer (FastAPI), Business Logic Layer (MySQL Stored Procedures, Functions, and Triggers), and Data Access Layer (PyMySQL). This approach ensures modularity, scalability, and separation of concerns.

Key features include automated order processing, real-time inventory management through import/export logging, supplier coordination, payment tracking, and customer loyalty point management. The database schema implements strict normalization, referential integrity through foreign keys, and business rule enforcement via CHECK constraints and triggers.

The system demonstrates how database-driven business logic can reduce backend complexity while ensuring data consistency, integrity, and automated workflows. This report covers problem definition, system architecture, database design (ERD, schema, constraints), implementation details, and future recommendations.

Keywords: Database Management, MySQL, FastAPI, Beer Distribution, Inventory Management, RESTful API, Business Logic Automation

Contents

List of Figures

List of Tables

Listings

- Link Slide: **Click here to open the Canva slide.**

1 Project Introduction

1.1 Problem Definition

In today's digital economy, the use of advanced management systems to improve operational efficiency and business oversight has emerged as a key element in boosting competitiveness and overall success. The **Beer Distribution System** project is designed to modernize distribution operations, particularly within the beverage industry.

This system has been built using **PyCharm** and **MySQL** as the database foundation to guarantee reliability and the potential for future growth. The system aims to digitize distribution workflows and automate essential tasks in the beer supply chain, featuring core database tables such as:

- **Customers** – Client information and loyalty tracking
- **Orders** – Transaction records and order lifecycle
- **Products** – Beer catalog and pricing
- **Suppliers** – Procurement and contract management
- **Inventory** – Warehouse stock and audit tracking

These capabilities are intended to enhance operational efficiency, strengthen supply chain coordination, and maximize business performance by simplifying distribution processes and effectively managing data.

1.2 Key System Functions

The Beer Distribution System supports five core business functions:

1.2.1 Customer Management

The system allows for the creation, authorization, and management of customer accounts, including details such as contact information and loyalty points. This access control ensures efficient customer relationship management, optimizing service delivery and enhancing customer retention.

1.2.2 Order Processing and Management

The system automates the process of receiving and handling customer orders, including recording order details, total amounts, tax, shipping costs, and status in real-time. This optimizes the distribution workflow, improves delivery efficiency, and enhances the customer experience.

1.2.3 Supplier Coordination

The system manages supplier data and supply agreements, tracking quantities, supply dates, and contract terms to ensure a steady flow of products. This enhances procurement efficiency and supports effective supply chain coordination.

1.2.4 Inventory Tracking and Logging

The system automates inventory management by recording imports, exports, and quantity changes through detailed logs. This functionality supports accurate stock monitoring, reduces discrepancies, and provides insights for inventory optimization.

1.2.5 Payment and Delivery Management

The system handles payment methods and statuses, alongside delivery dates, ensuring accurate financial tracking and timely order fulfillment, thereby improving cash flow management and customer satisfaction.

2 Overview of the Architecture

2.1 General Architecture

The Beer Distribution System follows a modern three-layer architecture pattern designed to ensure modularity, scalability, and maintainability. Figure 1 illustrates the overall system design.

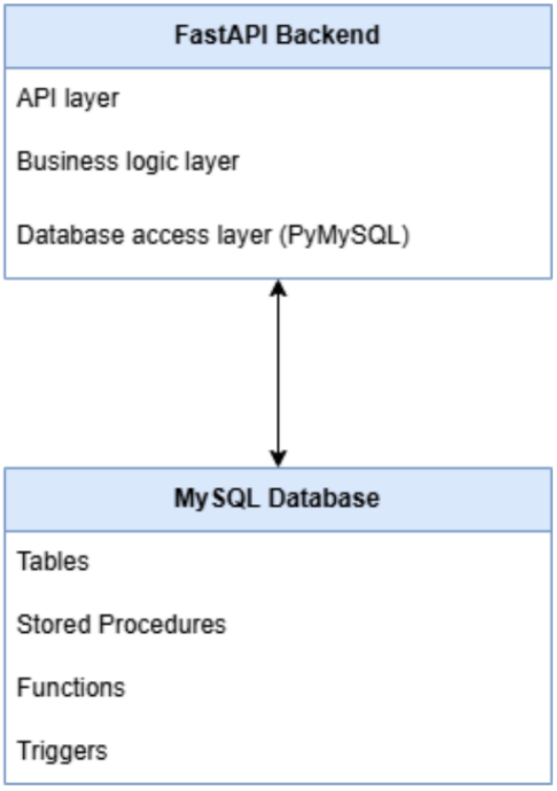

Figure 1: Three-Layer System Architecture

2.1.1 API Layer – FastAPI

This layer exposes RESTful API endpoints to external clients or system operators. It handles:

- Receiving HTTP requests
- Validating input parameters
- Forwarding requests to the Business Logic layer
- Returning JSON responses

Example endpoints include:

- `/add-customer`
- `/add-order`
- `/add-order-detail`
- `/process-exim`
- `/get-stock`

The API Layer does *not* contain business rules; it only coordinates logic and communicates with deeper layers.

2.1.2 Business Logic Layer (MySQL Stored Procedures, Functions, Triggers)

In this system, most core business logic is implemented directly inside MySQL to ensure consistency, performance, and transactional safety.

This layer includes:

Stored Procedures Handle complex operations such as:

- Generating new customer and order IDs
- Creating new orders
- Adding order details
- Handling import/export inventory movements
- Updating order statuses

Examples: `AddNewCustomer`, `AddNewOrder`, `AddOrderDetail`, `ProcessExIm`

Functions Return calculated values (read-only logic):

- `fn_get_order_total()` – calculates total value of an order
- `fn_get_stock()` – computes actual stock (import – export)
- `fn_get_customer_total_spent()`
- `fn_get_customer_order_count()`

Triggers Automatically ensure data integrity:

- Update inventory quantities after import/export
- Recalculate order totals
- Add loyalty points when an order is paid
- Update audit timestamps for inventory changes

By keeping business rules inside MySQL, the system guarantees consistency even if future systems (e.g., mobile app, web app) also connect to the database.

2.1.3 Data Access Layer (Python–MySQL Communication)

This layer provides a secure method for the API to interact with the database via:

- `pymysql` connections
- Parameterized queries
- Stored procedure calls (`cursor.callproc()`)

The DAL is responsible only for:

- Opening/closing database connections
- Running SQL or executing stored procedures
- Returning results to the API layer

No business logic is implemented here.

2.1.4 Database (MySQL)

The database contains all business entities:

- Customers
- Orders
- OrderDetail
- Products
- Inventory
- Ex_Im (import/export logs)
- Suppliers
- Product_Supplier

The schema supports normalization, foreign keys, cascading deletes, and consistent auditing of inventory.

2.2 Workflows

The following are example workflows demonstrating how data flows through the system:

2.2.1 Add New Customer

User → POST /add-customer

→ FastAPI → CALL AddNewCustomer

→ MySQL:

- Generate CUSxxx
- Insert into Customers
- Select new_customer_id

← FastAPI return JSON

2.2.2 Add New Order

User → POST /add-order

→ FastAPI → CALL AddNewOrder

→ MySQL:

- Generate ORDxxx
- Insert into Orders
- Select new_order_id

← FastAPI return JSON

2.2.3 Add New Order Details

User → POST /add-order-detail

→ FastAPI → CALL AddOrderDetail

→ MySQL:

- Insert OrderDetail
- Update total_amount by using fn_get_order_total()
- Trigger trg_update_order_total (if exist)

2.2.4 Update Order Status

User → PUT /update-order-status

→ FastAPI → CALL UpdateOrderStatus

→ MySQL:

- Update Orders
- Trigger trg_update_customer_loyalty if order status is Paid

3 Database Design

3.1 Entity-Relationship Model

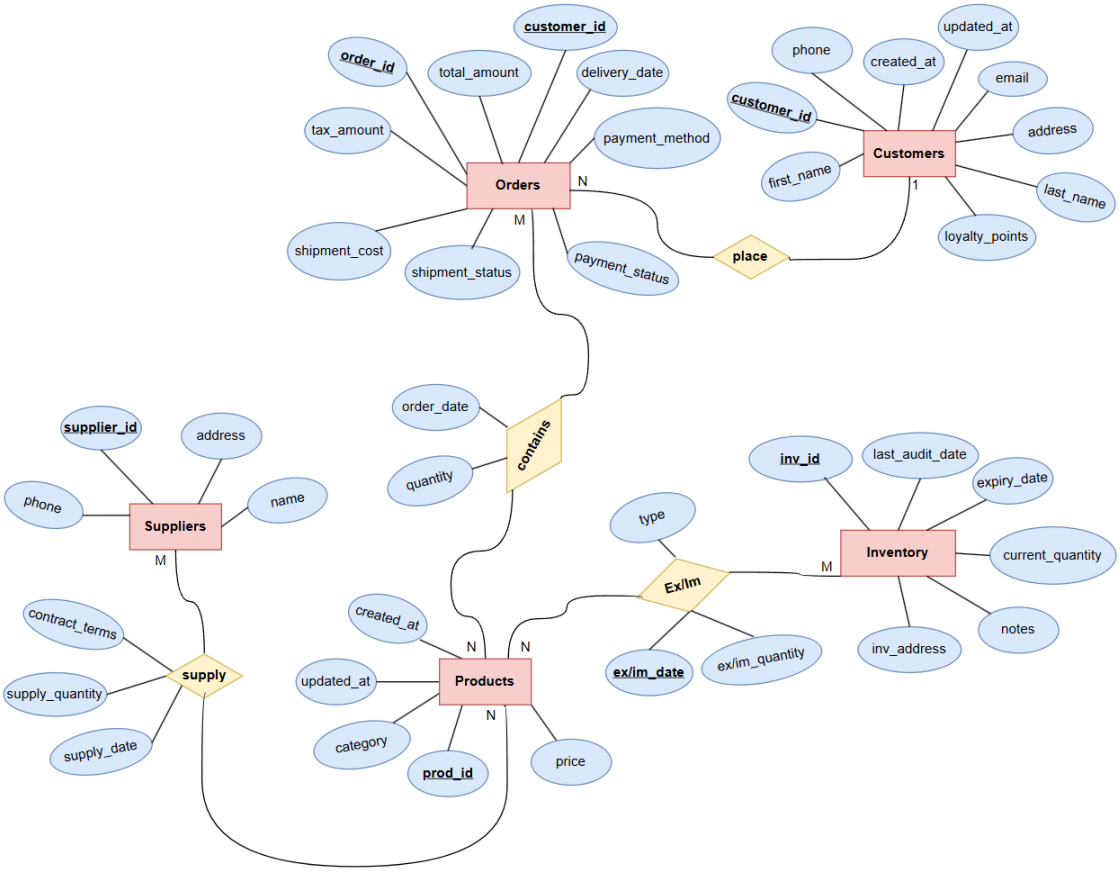

Figure 2: Entity-Relationship Diagram (ERD)

The Entity–Relationship (ER) model represents the logical structure of the Beer Distribution System. It defines how real-world business objects such as **Customers, Orders, Products, Suppliers**, and warehouses (**Inventory**) interact with each other in the system.

Each entity reflects a key data component of the business, while the relationships illustrate how transactions and workflows are connected across the supply chain. This model ensures that the database is logically organized, normalized, and capable of supporting the system's main operations efficiently.

3.1.1 Core Entities

The Beer Distribution System is structured around five main entities:

1. **Customers** – represent the buyers who place orders for beer products. This entity stores key identification and contact information to support customer relationship management and loyalty tracking.

2. **Orders** – record all transactions made by customers, including payment and shipment details, allowing the system to track the full order lifecycle from purchase to delivery.
3. **Products** – represent all beer items distributed or sold by the company. This entity serves as the central reference for other operations such as ordering, supplying, and storing.
4. **Suppliers** – contain information about companies or distributors that provide beer products to the system, supporting procurement and contract management.
5. **Inventory** – represent the warehouses where products are stored, tracking stock levels, expiration dates, and audits to ensure proper supply and storage management.

Together, these entities form the foundation of the ER model, supporting all major business functions—from purchasing and supply management to storage and sales distribution.

3.1.2 Relationships

Table 1 summarizes the key relationships and their mapped system functions.

Table 1: Relationships and Corresponding System Functions

Relationship	Detailed Description	Mapped Function
place (Customers → Orders) 1:N	Represents the relationship between customers and orders: one customer can place multiple orders, but each order belongs to only one customer. The system uses this relationship to identify who places the order, track purchase history, and loyalty points.	Order Processing and Management – processing orders based on customer information.
contains (Orders → Products) M:N	Indicates that one order can contain multiple products, and a product can appear in many different orders. The system records product details, quantity, price, tax, and total amount in each order.	Order Processing and Management – recording and processing detailed order items.
supply (Suppliers → Products) M:N	Represents the relationship between suppliers and products: one supplier can provide many products, and a product can be supplied by multiple suppliers. Includes attributes like `supply_quantity`, `supply_date`, `contract_terms` to manage supply contracts and stock quantity.	Supplier Coordination – managing supplier data, contracts, and supply chain.
Ex/Im (Products → Inventory) M:N	Records import/export (stock-in/out) operations between products and warehouse. Each log saves `ex/im_date`, `type` (Import/Export), and `quantity`. Helps the system track inventory flow and maintain stock accuracy.	Inventory Tracking and Logging – supporting warehouse management and stock movement tracking.

3.2 Transforming ERD to Tables

3.2.1 Entity Tables

The ERD entities are transformed into the following relational tables:

- **Customers** (PK (**customer_id**), first_name, last_name, email, phone, address, loyalty_points, created_at, updated_at)
- **Orders** (PK (**order_id**), FK (**customer_id**), total_amount, tax_amount, shipment_cost, payment_method, payment_status, shipment_status, delivery_date)
- **Products** (PK (**prod_id**), name, category, price, created_at, updated_at)

- **Suppliers** (PK (**supplier_id**), name, address, phone)
- **Inventory** (PK (**inv_id**), inv_address, expiry_date, current_quantity, notes, last_audit_date)

3.2.2 Relationship Tables

The many-to-many relationships are represented by junction tables:

- **Product_Supplier** (PK (**supplier_id, prod_id**), contract_terms, supply_quantity, supply_date)
- **OrderDetail** (PK (**order_id, prod_id**), quantity, order_date)
- **Ex_Im** (PK (**prod_id, inv_id, ex_im_date**), type, ex_im_quantity)

3.3 Constraints/Rules, and Norms

The database schema of the Beer Distribution System is designed with a series of constraints to ensure data integrity, consistency, and logical accuracy. These constraints prevent invalid data entry, preserve relationships between entities, and reflect real-world business rules such as unique customers, positive stock quantities, and valid payment or shipment statuses.

Each constraint was applied intentionally to represent the operational logic of the system and maintain normalization across all tables.

3.3.1 Primary Key Constraints

Purpose: Primary Keys (PK) uniquely identify each record in a table. They ensure that no two rows contain the same identifier, enabling accurate referencing across related entities.

Application in the System:

- `Customers(customer_id)` uniquely identifies each customer.
- `Orders(order_id)` identifies each purchase transaction.
- `Products(prod_id)` defines each product in the catalog.
- `Suppliers(supplier_id)` and `Inventory(inv_id)` ensure distinct warehouse and supplier records.
- Relationship tables such as `OrderDetail`, `Product_Supplier`, and `Ex_Im` use composite primary keys (e.g., `(order_id, prod_id)`) to prevent duplication in many-to-many relationships.

Reasoning: This guarantees the uniqueness and traceability of every record—a fundamental principle of relational database design.

3.3.2 Foreign Key Constraints

Purpose: Foreign Keys (FK) maintain referential integrity between related tables. They ensure that every reference points to a valid record in its parent table.

Application in the System:

- `Orders(customer_id)` → `Customers(customer_id) ON DELETE CASCADE`
- `OrderDetail(order_id)` → `Orders(order_id) ON DELETE CASCADE`
- `OrderDetail(prod_id)` → `Products(prod_id) ON DELETE CASCADE`
- `Product_Supplier(supplier_id)` → `Suppliers(supplier_id) ON DELETE CASCADE`
- `Product_Supplier(prod_id)` → `Products(prod_id) ON DELETE CASCADE`
- `Ex_Im(prod_id)` → `Products(prod_id) ON DELETE CASCADE`
- `Ex_Im(inv_id)` → `Inventory(inv_id) ON DELETE CASCADE`

ON DELETE CASCADE: Used to automatically remove dependent records when a parent record is deleted (e.g., deleting a customer deletes all their orders). This maintains database consistency and avoids orphan records.

Practical Justification: In real-world systems, when an entity (like a customer or product) is removed, all dependent data must also be deleted or archived to keep the database clean and consistent. For example:

- Deleting a Customer → removes all related Orders and OrderDetail records.
- Deleting a Product → removes related records in OrderDetail, Product_Supplier, and Ex_Im.
- Deleting a Supplier → removes all linked supply contracts in Product_Supplier.

3.3.3 UNIQUE Constraints

Purpose: UNIQUE ensures that certain data fields—such as emails or phone numbers—cannot be duplicated, which is essential for accurate identification.

Application:

- `Customers.email` and `Customers.phone` are unique.
- `Suppliers.phone` is unique.

Reasoning: This prevents data duplication and ensures that each customer or supplier can be contacted individually. It also avoids confusion in CRM systems and supports reliable authentication if needed.

3.3.4 CHECK Constraints

Purpose: CHECK constraints ensure that column values comply with valid business rules and logical boundaries.

Examples and Justification:

- `Customers: loyalty_points >= 0` (Points cannot be negative)

- `Orders: total_amount >= 0, tax_amount >= 0, shipment_cost >= 0` (No negative monetary values)
- `Orders: payment_method IN ('Bank Transfer','Credit Card','Cash')` (Restrict to valid payment methods)
- `Orders: shipment_status IN ('Pending','Confirmed','In Delivery','Delivered','` (Restrict to real shipment states)
- `OrderDetail: quantity > 0` (Must order at least one unit)
- `Product_Supplier: supply_quantity > 0` (No zero-supply contracts)
- `Ex_Im: ex_im_quantity > 0` (Stock transactions must move positive quantities)
- `Products: price >= 0` (Prevent invalid product pricing)

Reasoning: These constraints enforce logical correctness of business operations, prevent negative or invalid values, and simplify validation at the database level rather than relying only on application logic.

3.3.5 DEFAULT and Timestamp Constraints

Purpose: DEFAULT values ensure that certain columns automatically receive valid data if not explicitly provided. Timestamps such as `CURRENT_TIMESTAMP` are used for tracking creation and update times.

Application:

- `created_at` and `updated_at` in `Customers` and `Products` → track changes automatically.
- `Ex_Im.ex_im_date`, `OrderDetail.order_date`, and `Product_Supplier.supply_date` default to the current date/time.

Reasoning: This simplifies data entry and ensures auditability—every record is traceable in terms of when it was created or modified.

3.3.6 ON DELETE CASCADE

Purpose: Maintains referential integrity by automatically deleting dependent records when a parent record is deleted.

Examples:

- Deleting a Customer → removes all related `Orders` and `OrderDetail` records.
- Deleting a Product → removes related records in `OrderDetail`, `Product_Supplier`, and `Ex_Im`.
- Deleting a Supplier → removes all linked supply contracts.

Reasoning: This ensures no orphan records remain in relationship tables. It reflects realistic business behavior — when a supplier or product is removed, all dependent transactions should be cleared or archived accordingly.

Alternative: If historical tracking is required, ON DELETE SET NULL or RESTRICT could be used instead of CASCADE.

3.3.7 ENUM Constraints

Purpose: To limit the input values of certain attributes to a predefined list, avoiding invalid entries.

Example: In `Ex_Im`, `type ENUM('Export','Import')` ensures that transactions are categorized as either an Import or Export.

Reasoning: This simplifies data validation and guarantees that all inventory movements have a valid, consistent classification.

3.3.8 Business Logic Summary

Table 2: Constraints and Business Logic Summary

Constraint Type	Purpose
PRIMARY KEY	Ensure unique identification
FOREIGN KEY	Maintain referential integrity
UNIQUE	Prevent duplication of key info
CHECK	Enforce logical conditions
DEFAULT / TIMESTAMP	Automate data population
ENUM	Restrict data to valid options
ON DELETE CASCADE	Auto-clean dependent data

3.3.9 Conclusion

The applied constraints collectively enforce both technical integrity and business logic of the Beer Distribution System. They ensure that data remains accurate, non-redundant, and consistent. Invalid inputs (e.g., negative prices, undefined statuses) are blocked. All relationships (Customer–Order, Product–Supplier, Product–Inventory) stay synchronized.

By combining structural rules (PK, FK, UNIQUE) with logical rules (CHECK, ENUM), the system achieves a fully normalized and business-safe database design.

3.4 Database Schema Tables

The complete database schema is presented in the following tables.

3.4.1 Customers Table

Table 3: Customers Table Schema

Column Name	Data Type	Constraints
customer_id	VARCHAR(10)	PRIMARY KEY
first_name	VARCHAR(50)	NOT NULL
last_name	VARCHAR(50)	NOT NULL
email	VARCHAR(100)	UNIQUE, NOT NULL
phone	VARCHAR(15)	UNIQUE
address	VARCHAR(255)	
loyalty_points	INT	DEFAULT 0, CHECK (...)
created_at	DATETIME	CURRENT_TIMESTAMP
updated_at	DATETIME	CURRENT_TIMESTAMP UPDATE

3.4.2 Orders Table

Table 4: Orders Table Schema

Column Name	Data Type	Constraints
order_id	VARCHAR(10)	PRIMARY KEY
customer_id	VARCHAR(10)	FK REFERENCES Customers
total_amount	DECIMAL(10,2)	NOT NULL, DEFAULT 0
tax_amount	DECIMAL(10,2)	DEFAULT 0, CHECK (...)
shipment_cost	DECIMAL(10,2)	DEFAULT 0, CHECK (...)
payment_method	VARCHAR(50)	CHECK (payment_method IN...)
payment_status	VARCHAR(50)	CHECK (payment_status IN...)
shipment_status	VARCHAR(50)	CHECK (shipment_status IN...)
delivery_date	DATE	

3.4.3 Products Table

Table 5: Products Table Schema

Column Name	Data Type	Constraints
prod_id	VARCHAR(10)	PRIMARY KEY
name	VARCHAR(100)	NOT NULL
category	VARCHAR(50)	
price	DECIMAL(10,2)	NOT NULL, CHECK(...)
created_at	DATETIME	CURRENT_TIMESTAMP
updated_at	DATETIME	CURRENT_TIMESTAMP UPDATE

3.4.4 Suppliers Table

Table 6: Suppliers Table Schema

Column Name	Data Type	Constraints
supplier_id	VARCHAR(10)	PRIMARY KEY
name	VARCHAR(100)	NOT NULL
address	VARCHAR(255)	
phone	VARCHAR(15)	UNIQUE

3.4.5 Inventory Table

Table 7: Inventory Table Schema

Column Name	Data Type	Constraints
inv_id	VARCHAR(10)	PRIMARY KEY
inv_address	VARCHAR(255)	NOT NULL
expiry_date	DATE	
current_quantity	INT	DEFAULT 0, CHECK(...)
notes	TEXT	
last_audit_date	DATE	

3.4.6 Product_Supplier Table

Table 8: Product_Supplier Relationship Table

Column Name	Data Type	Constraints
supplier_id	VARCHAR(10)	FK REFERENCES Suppliers
prod_id	VARCHAR(10)	FK REFERENCES Products
contract_terms	TEXT	
supply_quantity	INT	CHECK (supply_quantity > 0)
supply_date	DATE	CURRENT_TIMESTAMP
PRIMARY KEY (supplier_id, prod_id)		

3.4.7 OrderDetail Table

Table 9: OrderDetail Relationship Table

Column Name	Data Type	Constraints
order_id	VARCHAR(10)	FK REFERENCES Orders
prod_id	VARCHAR(10)	FK REFERENCES Products
quantity	INT	NOT NULL, CHECK(quantity > 0)
order_date	DATE	CURRENT_TIMESTAMP
PRIMARY KEY (order_id, prod_id)		

3.4.8 Ex_Im Table

Table 10: Ex_Im (Import/Export Log) Table

Column Name	Data Type	Constraints
prod_id	VARCHAR(10)	FK REFERENCES Products
inv_id	VARCHAR(10)	FK REFERENCES Inventory
type	ENUM('Export', 'Import')	NOT NULL
ex_im_date	DATE	CURRENT_TIMESTAMP
ex_im_quantity	INT	NOT NULL, CHECK > 0
PRIMARY KEY (prod_id, inv_id, ex_im_date)		

3.4.9 Database Summary

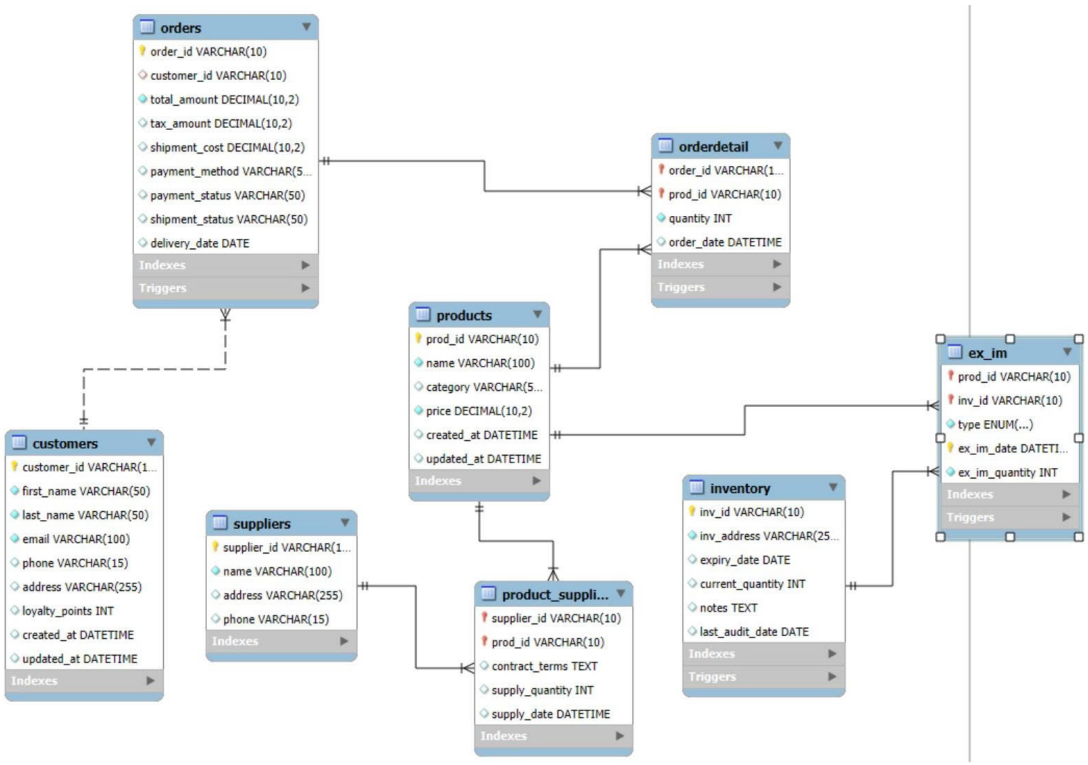

Figure 3: Database Schema Summary Diagram

4 Implementation

This chapter describes how the Beer Distribution Management System was implemented using a MySQL database, a FastAPI backend, and supporting Python libraries. The implementation focuses on three main components: the database layer, the API layer, and the libraries/tools used to connect them.

4.1 MySQL Implementation

The MySQL database is the core of the system, storing all operational data and enforcing business rules. The database is designed using a relational model with strong use of stored procedures, functions, and triggers to ensure data integrity and automate business logic.

4.1.1 Database Tables

The system uses several relational tables organized into two categories:

Core Entities:

- **Customers** – Stores customer information and loyalty points.
- **Products** – Contains product data including categories and pricing.
- **Orders** – Represents high-level order information for each customer.
- **Inventory** – Tracks stock quantities for each warehouse.
- **Suppliers** – Suppliers providing products.

Relationships:

- Products ↔ Orders → via `OrderDetail`
- Products ↔ Inventory → via `Ex_Im`
- Products ↔ Suppliers → via `Product_Supplier`

Primary keys and foreign keys ensure strong referential integrity, using `ON DELETE CASCADE` to automate cleanup of related rows.

4.1.2 Stored Procedures

The system centralizes business logic inside MySQL using stored procedures. This ensures consistency and reduces backend complexity.

- **AddNewCustomer:** Automatically generates a new ID (CUSXXX), inserts customer, returns new ID.
- **AddNewOrder:** Generates new ID (ORDXXX) and inserts a new order header.
- **AddOrderDetail:** Inserts product items into an order and recalculates the order total using the function `fn_get_order_total`.
- **ProcessExIm:** Handles Import/Export operations:
  - Updates inventory quantity
  - Inserts a log entry into Ex_Im
- **UpdateOrderStatus:** Updates order payment and shipment status.

These procedures encapsulate logic so that the API only needs to call them without manually writing SQL business rules.

4.1.3 MySQL Functions

The system includes several helper functions used in triggers and API queries:

- `fn_get_order_total(order_id)`: Calculates order total based on quantities × product price.
- `fn_get_stock(prod_id)`: Returns real-time inventory (total import − total export).
- `fn_get_customer_total_spent(customer_id)`: Returns total amount spent by a customer.
- `fn_get_customer_order_count(customer_id)`: Counts how many orders the customer has placed.
- `fn_get_total_inventory()`: Returns the sum of stock quantities across all warehouses.

These functions simplify calculations and make triggers more readable.

4.1.4 Triggers

Triggers are used to enforce automatic system rules:

- **trg_update_inventory_after_exim:** Whenever a new import/export record is inserted, update the inventory accordingly.
- **trg_update_order_total:** Recalculates order total after inserting a new OrderDetail row.
- **trg_update_customer_loyalty:** Adds loyalty points when an order changes to Paid status.
- **trg_update_inventory_audit:** Automatically updates `last_audit_date` whenever an inventory row is updated.

Triggers ensure that business rules execute automatically with no manual intervention.

4.2 API Implementation (FastAPI)

The backend API is implemented using **FastAPI**, which exposes database functionality to external systems or future frontend components.

4.2.1 API Structure

The backend is organized into three layers:

1. **API Layer:** Defines HTTP endpoints and receives/sends JSON data.
2. **Business Logic Layer:** Minimal, since logic is mostly handled inside MySQL procedures and functions.
3. **Database Access Layer:** Handles database connections using PyMySQL, calls stored procedures, commits transactions, and returns results.

This 3-layer approach keeps the backend clean, modular, and easy to maintain.

4.2.2 API Endpoints

The system provides several REST API endpoints:

GET Endpoints

- `/customers` – Fetch all customers
- `/orders` – Fetch all orders
- `/products` – Fetch all products
- `/inventory` – Fetch inventory list
- `/get-stock(prod_id)` – Get real-time product stock

POST Endpoints

- `/add-customer` – Calls `AddNewCustomer`
- `/add-order` – Calls `AddNewOrder`
- `/add-order-detail` – Calls `AddOrderDetail`
- `/process-exim` – Calls `ProcessExIm`

PUT Endpoints

- `/update-order-status` – Calls `UpdateOrderStatus`

4.2.3 Database Calls from API

Each API endpoint follows the same pattern:

1. Open MySQL connection
2. Execute stored procedure or SQL
3. Commit
4. Return JSON response

4.3 Libraries and Tools

4.3.1 PyMySQL

- Connects Python to MySQL
- Allows calling stored procedures
- Returns results as dictionaries (DictCursor)
- Handles transactions, queries, and data retrieval

4.3.2 FastAPI

Chosen because:

- Extremely fast (ASGI-based)
- Automatic documentation
- Easy to write and maintain
- Good JSON support

4.3.3 Uvicorn

ASGI server used to run the FastAPI application.

4.3.4 MySQL Workbench

Used to:

- Design schema
- Test procedures, functions, triggers
- Manually inspect database state

5 Conclusion

The Beer Distribution Management System was successfully designed and implemented using a combination of MySQL, FastAPI, and Python. Through a structured three-layer backend architecture—API Layer, Business Logic Layer, and Database Access Layer—the system achieves modularity, maintainability, and clear separation of responsibilities.

The database implementation forms a strong foundation for the system. By using stored procedures, functions, and triggers, critical business logic is embedded directly into MySQL, ensuring consistency, automation, and data integrity without requiring complex backend logic. Inventory updates, order processing, customer loyalty management, and automated total calculations all operate reliably through database-level mechanisms.

The FastAPI backend provides a clean and modern interface for interacting with the database. With well-defined REST API endpoints, the system supports customer management, product retrieval, order creation, import/export handling, and real-time stock checking. The use of PyMySQL ensures smooth communication between the API and the database, while FastAPI's built-in documentation enhances usability and testing.

Overall, this project demonstrates how a practical, real-world business system can be built efficiently by leveraging database-driven business logic and a lightweight API layer. The resulting system is scalable, easy to extend, and suitable for future integration with a frontend user interface or external systems. The combination of robust data modeling, automated workflows, and clean API design provides a solid foundation for further development and real deployment scenarios.

6 References

1. FastAPI Documentation. Available at: https://fastapi.tiangolo.com
2. PyMySQL Documentation. Available at: https://pymysql.readthedocs.io/en/latest/
3. Uvicorn ASGI Server. Available at: https://www.uvicorn.org
4. MySQL 8.0 Reference Manual. Available at: https://dev.mysql.com/doc
5. Recharts Documentation. Available at: https://recharts.org

A Appendix A: Database Schema

This appendix contains the complete SQL script for creating the database schema, including all table definitions and sample data insertion.

```sql
/* Create the database */
CREATE DATABASE IF NOT EXISTS ecommerce;
/* Switch to the ecommerce database */
USE ecommerce;

/* Drop existing tables in correct order to avoid foreign key
   constraints */
DROP TABLE IF EXISTS Ex_Im;
DROP TABLE IF EXISTS OrderDetail;
DROP TABLE IF EXISTS Product_Supplier;
DROP TABLE IF EXISTS Orders;
DROP TABLE IF EXISTS Inventory;
DROP TABLE IF EXISTS Products;
DROP TABLE IF EXISTS Suppliers;
DROP TABLE IF EXISTS Customers;

-- ============================
-- Table: Customers
-- ============================
CREATE TABLE Customers (
    customer_id VARCHAR(10) PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    phone VARCHAR(15) UNIQUE,
    address VARCHAR(255),
    loyalty_points INT DEFAULT 0
        CHECK (loyalty_points >= 0),
    created_at DATETIME
        DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME
        DEFAULT CURRENT_TIMESTAMP
        ON UPDATE CURRENT_TIMESTAMP
);

-- ============================
-- Table: Orders
-- ============================
CREATE TABLE Orders (
    order_id VARCHAR(10) PRIMARY KEY,
    customer_id VARCHAR(10),
    total_amount DECIMAL(10,2) NOT NULL
        DEFAULT 0 CHECK (total_amount >= 0),
    tax_amount DECIMAL(10,2) DEFAULT 0
        CHECK (tax_amount >= 0),
    shipment_cost DECIMAL(10,2) DEFAULT 0
        CHECK (shipment_cost >= 0),
```

```
    payment_method VARCHAR(50)
        CHECK (payment_method IN
        ('Bank Transfer','Credit Card','Cash')),
    payment_status VARCHAR(50)
        CHECK (payment_status IN
        ('Pending','Paid','Failed')),
    shipment_status VARCHAR(50)
        CHECK (shipment_status IN
        ('Pending','Shipped','Delivered',
         'Returned')),
    delivery_date DATE,
    FOREIGN KEY (customer_id)
        REFERENCES Customers(customer_id)
        ON DELETE CASCADE
);

-- ============================
-- Table: Products
-- ============================
CREATE TABLE Products (
    prod_id VARCHAR(10) PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    category VARCHAR(50),
    price DECIMAL(10,2) NOT NULL
        CHECK (price >= 0),
    created_at DATETIME
        DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME
        DEFAULT CURRENT_TIMESTAMP
        ON UPDATE CURRENT_TIMESTAMP
);

-- ============================
-- Table: Suppliers
-- ============================
CREATE TABLE Suppliers (
    supplier_id VARCHAR(10) PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    address VARCHAR(255),
    phone VARCHAR(15) UNIQUE
);

-- ============================
-- Table: Inventory
-- ============================
CREATE TABLE Inventory (
    inv_id VARCHAR(10) PRIMARY KEY,
    inv_address VARCHAR(255) NOT NULL,
    expiry_date DATE,
    current_quantity INT DEFAULT 0
        CHECK (current_quantity >= 0),
```

```
    notes TEXT,
    last_audit_date DATE
);

-- ============================
-- Table: Product_Supplier
-- ============================
CREATE TABLE Product_Supplier (
    supplier_id VARCHAR(10),
    prod_id VARCHAR(10),
    contract_terms TEXT,
    supply_quantity INT
        CHECK (supply_quantity > 0),
    supply_date DATETIME
        DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (supplier_id, prod_id),
    FOREIGN KEY (supplier_id)
        REFERENCES Suppliers(supplier_id)
        ON DELETE CASCADE,
    FOREIGN KEY (prod_id)
        REFERENCES Products(prod_id)
        ON DELETE CASCADE
);

-- ============================
-- Table: OrderDetail
-- ============================
CREATE TABLE OrderDetail (
    order_id VARCHAR(10),
    prod_id VARCHAR(10),
    quantity INT NOT NULL
        CHECK (quantity > 0),
    order_date DATETIME
        DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (order_id, prod_id),
    FOREIGN KEY (order_id)
        REFERENCES Orders(order_id)
        ON DELETE CASCADE,
    FOREIGN KEY (prod_id)
        REFERENCES Products(prod_id)
        ON DELETE CASCADE
);

-- ==============================
-- Table: Ex_Im (Export/Import)
-- ==============================
CREATE TABLE Ex_Im (
    prod_id VARCHAR(10),
    inv_id VARCHAR(10),
    type ENUM('Export','Import') NOT NULL,
    ex_im_date DATETIME
```

```
        DEFAULT CURRENT_TIMESTAMP,
    ex_im_quantity INT NOT NULL
        CHECK (ex_im_quantity > 0),
    PRIMARY KEY (prod_id, inv_id,
                 ex_im_date),
    FOREIGN KEY (prod_id)
        REFERENCES Products(prod_id)
        ON DELETE CASCADE,
    FOREIGN KEY (inv_id)
        REFERENCES Inventory(inv_id)
        ON DELETE CASCADE
);

-- SHOW TABLES;

-- SELECT * FROM Customers;
-- SELECT * FROM Orders;
-- SELECT * FROM Products;
-- SELECT * FROM Suppliers;
-- SELECT * FROM Inventory;
-- SELECT * FROM Product_Supplier;
-- SELECT * FROM OrderDetail;
-- SELECT * FROM Ex_Im;
```

Listing 1: Database Schema

B Appendix B: Stored Procedures

This appendix contains all stored procedures used in the Beer Distribution System for business logic automation.

```
DROP PROCEDURE IF EXISTS AddNewCustomer;
DROP PROCEDURE IF EXISTS AddNewOrder;
DROP PROCEDURE IF EXISTS AddOrderDetail;
DROP PROCEDURE IF EXISTS ProcessExIm;
DROP PROCEDURE IF EXISTS UpdateOrderStatus;

-- Add new customer (auto generate ID with letters and numbers):
DELIMITER $$
CREATE PROCEDURE AddNewCustomer(
    IN p_first_name VARCHAR(50),
    IN p_last_name VARCHAR(50),
    IN p_email VARCHAR(100),
    IN p_phone VARCHAR(15),
    IN p_address VARCHAR(255)
)
BEGIN
    DECLARE v_customer_id VARCHAR(10);
    DECLARE v_next INT;

    SELECT IFNULL(MAX(
```

```
        CAST(SUBSTRING(customer_id, 4)
        AS UNSIGNED)), 0) + 1
    INTO v_next FROM Customers;

    SET v_customer_id = CONCAT('CUS',
        LPAD(v_next, 3, '0'));

    INSERT INTO Customers
    (customer_id, first_name, last_name,
     email, phone, address)
    VALUES(v_customer_id, p_first_name,
           p_last_name, p_email, p_phone,
           p_address);

    SELECT v_customer_id AS new_customer_id;
END$$
DELIMITER ;

-- Add new order:
DELIMITER $$
CREATE PROCEDURE AddNewOrder(
    IN p_customer_id VARCHAR(10),
    IN p_payment_method VARCHAR(50),
    IN p_tax_amount DECIMAL(10,2),
    IN p_shipment_cost DECIMAL(10,2),
    IN p_payment_status VARCHAR(50),
    IN p_shipment_status VARCHAR(50),
    IN p_delivery_date DATE
)
BEGIN
    DECLARE v_order_id VARCHAR(10);
    DECLARE v_next INT;

    SELECT IFNULL(MAX(
        CAST(SUBSTRING(order_id, 4)
        AS UNSIGNED)), 0) + 1
    INTO v_next FROM Orders;

    SET v_order_id = CONCAT('ORD',
        LPAD(v_next, 3, '0'));

    INSERT INTO Orders
    (order_id, customer_id,
     payment_method, payment_status,
     shipment_status, delivery_date)
    VALUES(v_order_id, p_customer_id,
           p_payment_method,
           p_payment_status,
           p_shipment_status,
           p_delivery_date);

```

```
    SELECT v_order_id AS new_order_id;
END$$
DELIMITER ;

-- Add order details:
DELIMITER $$
CREATE PROCEDURE AddOrderDetail(
    IN p_order_id VARCHAR(10),
    IN p_prod_id VARCHAR(10),
    IN p_quantity INT
)
BEGIN
    INSERT INTO OrderDetail
    (order_id, prod_id, quantity)
    VALUES(p_order_id, p_prod_id,
           p_quantity);

    UPDATE Orders
    SET total_amount =
        fn_get_order_total(p_order_id)
    WHERE order_id = p_order_id;
END$$
DELIMITER ;

-- Handle import or export of goods:
DELIMITER $$
CREATE PROCEDURE ProcessExIm(
    IN p_prod_id VARCHAR(10),
    IN p_inv_id VARCHAR(10),
    IN p_type ENUM('Export','Import'),
    IN p_exim_quantity INT
)
BEGIN
    -- Update inventory:
    IF p_type = 'Import' THEN
        UPDATE Inventory
        SET current_quantity =
            current_quantity +
            p_exim_quantity,
            last_audit_date =
            CURRENT_DATE()
        WHERE inv_id = p_inv_id;
    ELSEIF p_type = 'Export' THEN
        UPDATE Inventory
        SET current_quantity =
            current_quantity -
            p_exim_quantity,
            last_audit_date =
            CURRENT_DATE()
        WHERE inv_id = p_inv_id;
    END IF;
```

```
123
124     INSERT INTO Ex_Im
125     (prod_id, inv_id, type,
126      ex_im_quantity)
127     VALUES(p_prod_id, p_inv_id,
128            p_type, p_exim_quantity);
129 END$$
130 DELIMITER ;
131
132 -- Update payment and shipping status:
133 DELIMITER $$
134 CREATE PROCEDURE UpdateOrderStatus(
135     IN p_order_id VARCHAR(10),
136     IN p_payment_status VARCHAR(50),
137     IN p_shipment_status VARCHAR(50)
138 )
139 BEGIN
140     UPDATE Orders
141     SET payment_status = p_payment_status,
142         shipment_status = p_shipment_status
143     WHERE order_id = p_order_id;
144 END$$
145 DELIMITER ;
```

Listing 2: Stored Procedures Implementation

C Appendix C: Functions

This appendix contains all MySQL functions for calculations and data retrieval.

```
1  DROP FUNCTION IF EXISTS fn_get_order_total;
2  DROP FUNCTION IF EXISTS fn_get_customer_total_spent;
3  DROP FUNCTION IF EXISTS fn_get_stock;
4  DROP FUNCTION IF EXISTS fn_get_customer_order_count;
5  DROP FUNCTION IF EXISTS fn_get_total_inventory;
6
7  -- Calculate the total amount of one order:
8  DELIMITER $$
9  CREATE FUNCTION fn_get_order_total
10 (p_order_id VARCHAR(10))
11 RETURNS DECIMAL(10,2)
12 DETERMINISTIC
13 BEGIN
14     DECLARE v_total DECIMAL(10,2);
15
16     SELECT SUM(od.quantity * p.price)
17     INTO v_total
18     FROM OrderDetail od
19     JOIN Products p ON
20         od.prod_id = p.prod_id
21     WHERE od.order_id = p_order_id;
```

```
22
23      RETURN IFNULL(v_total, 0);
24  END$$
25  DELIMITER ;
26
27  -- Calculate the total amount spent by the customer:
28  DELIMITER $$
29  CREATE FUNCTION
30  fn_get_customer_total_spent
31  (p_customer_id VARCHAR(10))
32  RETURNS DECIMAL(10,2)
33  DETERMINISTIC
34  BEGIN
35      DECLARE v_total DECIMAL(10,2);
36
37      SELECT SUM(total_amount)
38      INTO v_total
39      FROM Orders
40      WHERE customer_id = p_customer_id;
41
42      RETURN IFNULL(v_total, 0);
43  END$$
44  DELIMITER ;
45
46  -- Retrieve the actual inventory (Imports - Exports):
47  DELIMITER $$
48  CREATE FUNCTION fn_get_stock
49  (p_prod_id VARCHAR(10))
50  RETURNS INT
51  DETERMINISTIC
52  BEGIN
53      DECLARE v_import INT;
54      DECLARE v_export INT;
55
56      SELECT IFNULL(SUM(ex_im_quantity),0)
57      INTO v_import
58      FROM Ex_Im
59      WHERE prod_id = p_prod_id AND
60            type = 'Import';
61
62      SELECT IFNULL(SUM(ex_im_quantity),0)
63      INTO v_export
64      FROM Ex_Im
65      WHERE prod_id = p_prod_id AND
66            type = 'Export';
67
68      RETURN v_import - v_export;
69  END$$
70  DELIMITER ;
71
72  -- Calculate the total number of orders from the customer:
```

```
DELIMITER $$
CREATE FUNCTION
fn_get_customer_order_count
(p_customer_id VARCHAR(10))
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE v_count INT;

    SELECT COUNT(*) INTO v_count
    FROM Orders
    WHERE customer_id = p_customer_id;

    RETURN IFNULL(v_count, 0);
END$$
DELIMITER ;

-- Calculate the total number of products currently in stock:
DELIMITER $$
CREATE FUNCTION fn_get_total_inventory()
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE v_total INT;

    SELECT SUM(current_quantity)
    INTO v_total FROM Inventory;

    RETURN IFNULL(v_total, 0);
END$$
DELIMITER ;
```

Listing 3: MySQL Functions Implementation

D Appendix D: Triggers

This appendix contains all database triggers for automated business logic enforcement.

```
DROP TRIGGER IF EXISTS trg_update_inventory_after_exim;
DROP TRIGGER IF EXISTS trg_update_order_total;
DROP TRIGGER IF EXISTS trg_update_customer_loyalty;
DROP TRIGGER IF EXISTS trg_update_inventory_audit;

-- Automatically update inventory upon inbound/outbound
   transactions:
DELIMITER $$
CREATE TRIGGER trg_update_inventory_after_exim
AFTER INSERT ON Ex_Im
FOR EACH ROW
BEGIN
    IF NEW.type = 'Import' THEN
```

```
        UPDATE Inventory
        SET current_quantity =
            current_quantity +
            NEW.ex_im_quantity
        WHERE inv_id = NEW.inv_id;
    ELSEIF NEW.type = 'Export' THEN
        UPDATE Inventory
        SET current_quantity =
            current_quantity -
            NEW.ex_im_quantity
        WHERE inv_id = NEW.inv_id;
    END IF;
END$$
DELIMITER ;

-- Automatically update the total order amount when adding
   OrderDetail:
DELIMITER $$
CREATE TRIGGER trg_update_order_total
AFTER INSERT ON OrderDetail
FOR EACH ROW
BEGIN
    UPDATE Orders
    SET total_amount =
        fn_get_order_total(NEW.order_id)
    WHERE order_id = NEW.order_id;
END$$
DELIMITER ;

-- Add loyalty points to customers upon completion of payment:
DELIMITER $$
CREATE TRIGGER trg_update_customer_loyalty
AFTER UPDATE ON Orders
FOR EACH ROW
BEGIN
    IF NEW.payment_status = 'Paid' AND
       OLD.payment_status <> 'Paid' THEN
        UPDATE Customers
        SET loyalty_points =
            loyalty_points +
            FLOOR(NEW.total_amount / 10)
        WHERE customer_id = NEW.customer_id;
    END IF;
END$$
DELIMITER ;

-- Update the inventory stocktaking date:
DELIMITER $$
CREATE TRIGGER
trg_update_inventory_audit
BEFORE UPDATE ON Inventory
```

```
FOR EACH ROW
BEGIN
    SET NEW.last_audit_date =
        CURRENT_DATE();
END$$
DELIMITER ;
```

Listing 4: Database Triggers Implementation

E Appendix E: FastAPI Backend

This appendix contains the complete FastAPI backend implementation for the Beer Distribution System.

```
import pymysql
from fastapi import FastAPI
import uvicorn

def get_connection():
    mydb = pymysql.connect(
        host="localhost",
        user="root",
        password="ahihidocho1652",
        database="beer",
        cursorclass=pymysql.cursors.DictCursor
    )
    return mydb

app = FastAPI(title="Beer Distribution System")

@app.get("/customers")
def get_customers():
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM customers")
    customers = cursor.fetchall()
    conn.close()
    return customers

@app.get("/orders")
def get_orders():
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM orders")
    orders = cursor.fetchall()
    conn.close()
    return orders

@app.get("/inventory")
def get_inventory():
    conn = get_connection()
```

```
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM inventory")
    inventory = cursor.fetchall()
    conn.close()
    return inventory

@app.get("/products")
def get_products():
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM products")
    products = cursor.fetchall()
    conn.close()
    return products

@app.post("/add-customer")
def add_customer(first_name: str, last_name: str, email: str,
    phone: str, address: str):
    try:
        conn = get_connection()
        cursor = conn.cursor()
        cursor.callproc("AddNewCustomer", (first_name, last_name
    , email, phone, address))

        result = cursor.fetchall()

        conn.commit()
        cursor.close()
        conn.close()
        return {
            "message": "Customer added successfully",
            "new_id": result[0]["new_customer_id"] if result
    else None
        }
    except Exception as e:
        return {"error": str(e)}

@app.post("/add-order")
def add_order(customer_id: str, payment_method: str, tax_amount:
    float, shipment_cost: float,
              payment_status: str, shipment_status: str,
    delivery_date: str):
    try:
        conn = get_connection()
        cursor = conn.cursor()
        cursor.callproc("AddNewOrder", (
            customer_id, payment_method, tax_amount,
    shipment_cost,
            payment_status, shipment_status, delivery_date
        ))

```

```
        result = cursor.fetchall()
        conn.commit()
        cursor.close()
        conn.close()
        return {
            "message": "Order added successfully",
            "new_id": result[0]["new_order_id"] if result else None
        }
    except Exception as e:
        return {"error": str(e)}

@app.post("/add-order-detail")
def add_order_detail(order_id: str, prod_id: str, quantity: int):
    try:
        conn = get_connection()
        cursor = conn.cursor()
        cursor.callproc("AddOrderDetail", (order_id, prod_id, quantity))
        conn.commit()
        cursor.close()
        conn.close()
        return {
            "message": f"Added product {prod_id} (x{quantity}) to order {order_id}"
        }
    except Exception as e:
        return {"error": str(e)}

@app.post("/process-exim")
def process_exim(prod_id: str, inv_id: str, type: str, exim_quantity: int):
    try:
        conn = get_connection()
        cursor = conn.cursor()
        cursor.callproc("ProcessExIm", (prod_id, inv_id, type, exim_quantity))
        conn.commit()
        cursor.close()
        conn.close()
        return {
            "message": f"{type} processed successfully for product {prod_id}"
        }
    except Exception as e:
        return {"error": str(e)}

@app.get("/get-stock")
def get_stock(prod_id: str):
    try:
```

```
        conn = get_connection()
        cursor = conn.cursor()

        # Call function fn_get_stock(product_id)
        cursor.execute("SELECT fn_get_stock(%s) AS current_stock
    ", (prod_id,))
        result = cursor.fetchone()
        cursor.close()
        conn.close()
        return {
            "product_id": prod_id,
            "current_stock": result["current_stock"] if result
    else None
        }
    except Exception as e:
        return {"error": str(e)}

@app.put("/update-order-status")
def update_order_status(order_id: str, payment_status: str,
    shipment_status: str):
    try:
        conn = get_connection()
        cursor = conn.cursor()
        cursor.callproc("UpdateOrderStatus", (order_id,
    payment_status, shipment_status))
        conn.commit()
        cursor.close()
        conn.close()
        return {
            "message": f"Order {order_id} updated successfully"
        }
    except Exception as e:
        return {"error": str(e)}

if __name__ == "__main__":
    uvicorn.run("business_test:app", host="localhost",
    port=5000, reload=True)
```

Listing 5: FastAPI Backend Implementation